



1

Agenda

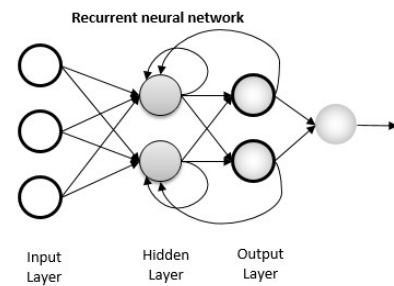
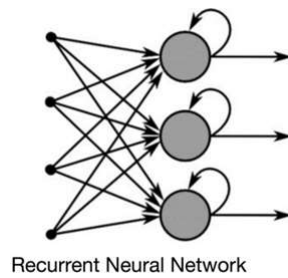
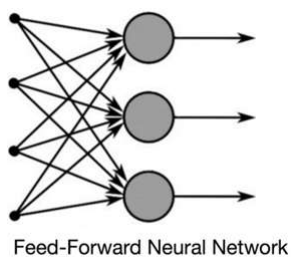
- Quick intro to sequence processing with RNN, LSTM, GRU, Applications
- Long range dependency
- What is attention?
- Transformer Architecture – Self-attention, Multi-head attention
- Transformer types
- BERT & GPT

2

Sequence Processing Models

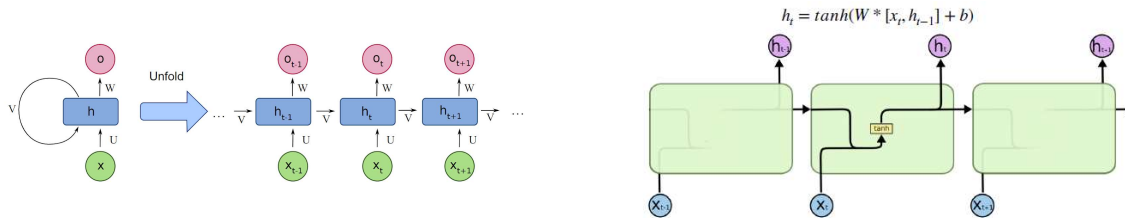
3

Model Architecture – FFN vs. RNN



4

Model Architecture – Vanilla RNN



- Vanilla RNN Cell
- Rolled/Folded vs. Time-unrolled/Unfolded version
- Parameter sharing over different time steps (essentially the same RNN cell).

5

Vanishing/Exploding Gradients

- In RNN, calculated gradient is result of multiplication of multiple individual derivatives
- As the sequence size increases, these multiplication can lead to either vanishing gradients (due to multiplying multiple less than one values).
- Or exploding gradient (due to multiplying multiple greater than one values).
- Gradient values can be contained to some extent by gradient clipping (capping lower/higher values to a limit).
- Vanishing gradient can be contained by using ReLU like activation functions to some extent (constant gradient 1).
- **Unable to model long-range dependency!**

6

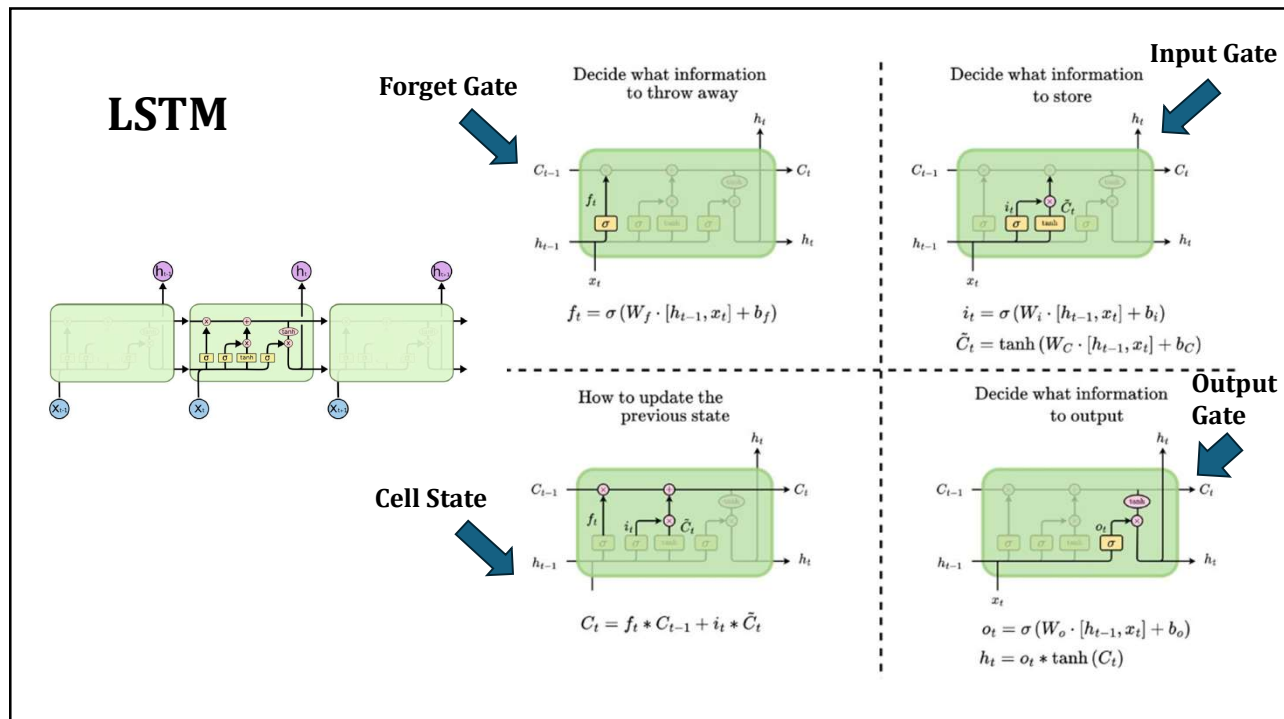
Inability to retain long-term memory (long range dependency problem)

- We saw that the more backward in the sequence we go, the less they contribute to parameter update/learning (due to the multiplication of small values => vanishing gradient).
- This essentially means that models can not retain long-term dependencies and will be affected/driven by recent trends mostly.



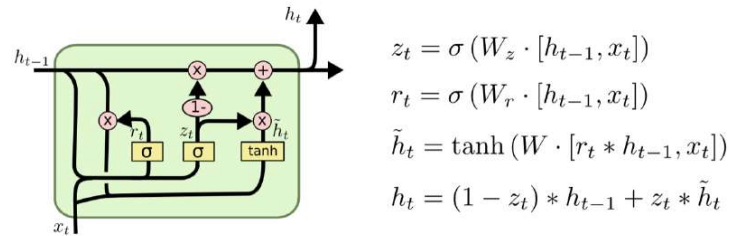
- The **child** who played football in the park all through the afternoon **was** tired.
- The **children** who played football in the park all through the afternoon **were** tired.

7



8

GRU

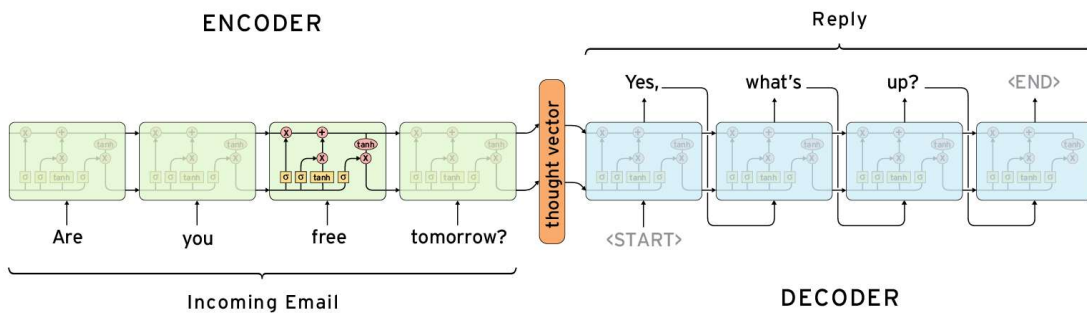


Motivation behind GRU was that LSTM has too many parameters which might not be necessarily needed.

- GRU achieves that by combining multiple components into one (not concatenating, simply one component takes care of both the ideas).
- **Forget and Input gates are combined into "Update gate" z_t**
- Cell state and hidden state are same.
- Despite being much simpler than LSTM in terms of parameter space, GRU have shown good results in many use cases (and faster to train).

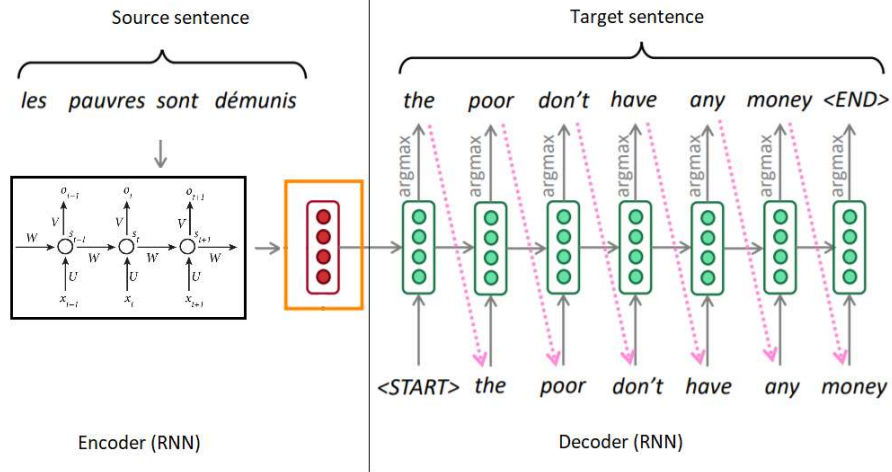
9

Question Answering



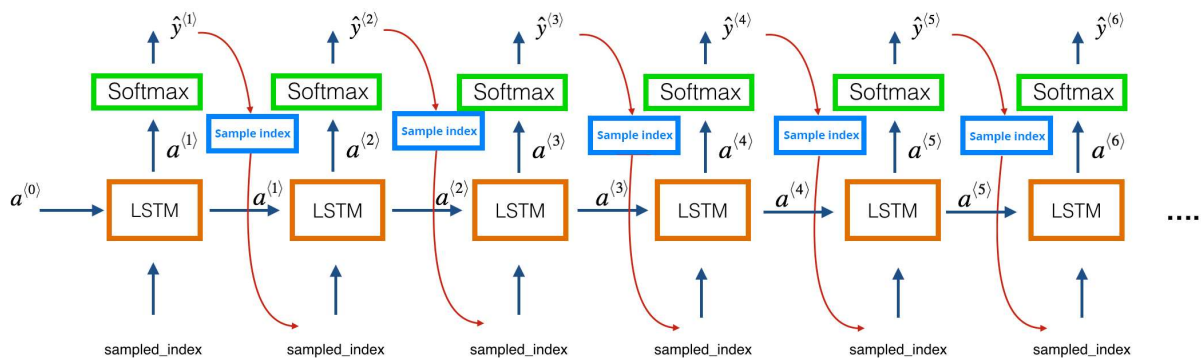
10

Language Translation



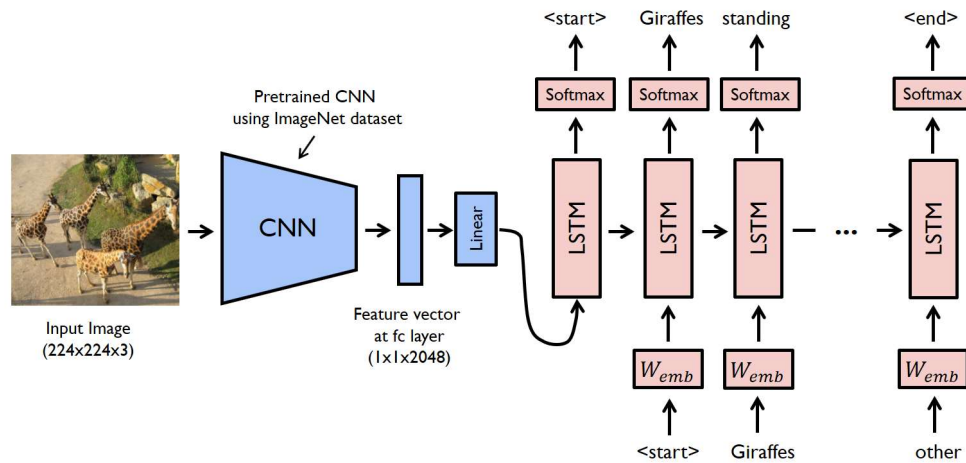
11

Application – Music Generation



12

Image Captioning



13

Understanding language can be challenging

The teacher's book?

The teacher taught the student with the book.

The student's book?

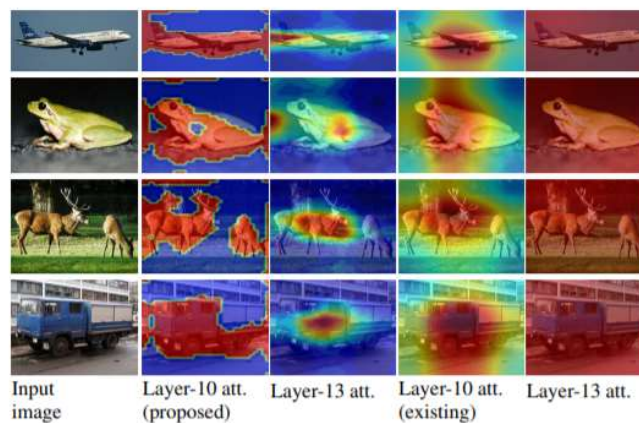
14

Attention

- Attention is one of the most influential ideas in the Deep Learning.
- In neural networks, attention is a technique that mimics cognitive attention.
- The effect enhances some parts of the input data while diminishing other parts — the thought being that the network should devote more focus to that small but important part of the data.
- Learning which part of the data is more important than others depends on the context and is trained by gradient descent.
- Even though this mechanism is now used in various problems like image captioning and others, it was initially designed in the context of Neural Machine Translation using Seq2Seq Models.

15

Visual Attention – Pay focus to certain part of the input data



16

Transformer & Attention

The Transformer neural network architecture proposed by Vaswani et al. (2017).

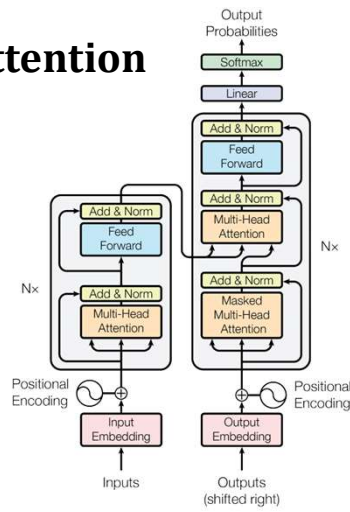


Figure 1: The Transformer - model architecture.

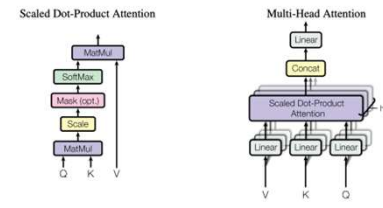


Figure 2: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel.

- Scale efficiently
- Parallel process
- Attention to input meaning

Vaswani et al. (2017): Attention is all you need

<https://papers.nips.cc/paper/7181-attention-is-all-you-need.pdf>

17

Transformer & Attention

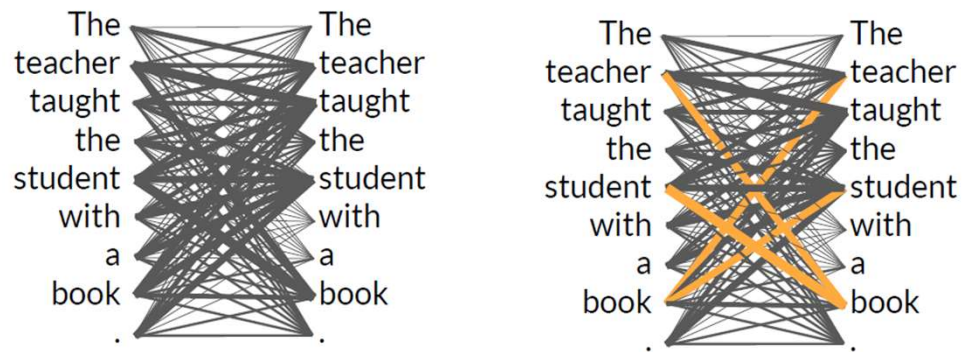
The teacher taught the student with the book.

The teacher taught the student with the book.

The teacher taught the student with the book.

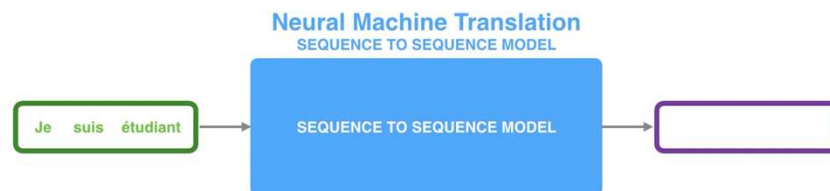
18

Self-Attention



19

Rough overview of seq2seq model



Source: <https://jalammar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/>

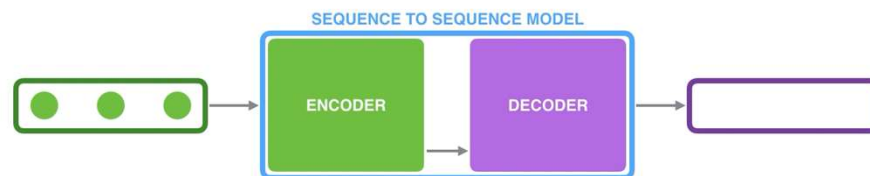
20

Encoder – Decoder (with Attention) for Sequence Processing

- The attention mechanism was introduced to improve the performance of the encoder-decoder model for machine translation.
- The idea behind the attention mechanism was to permit the Decoder to utilize the most relevant parts of the input sequence in a flexible manner, by a weighted combination of all of the Encoded input vectors, with the most relevant vectors being attributed the highest weights.

21

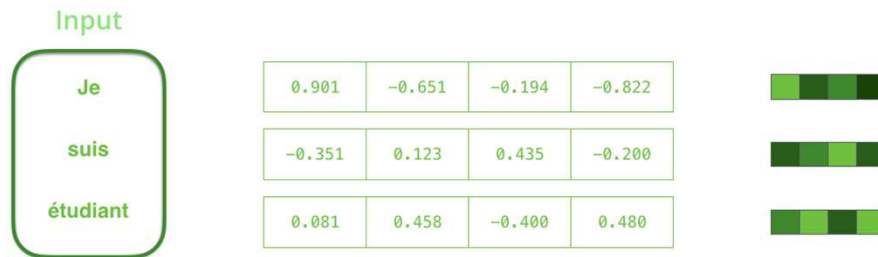
Encoder Decoder At work inside



Source: <https://jalammar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/>

22

Starting at input to embedded representation



Source: <https://jalammar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/>

23

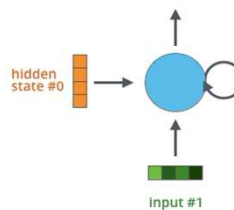
This input goes to time distributed RNNs

Recurrent Neural Network

Time step #1:
An RNN takes two input vectors:

hidden
state #0

input vector #1

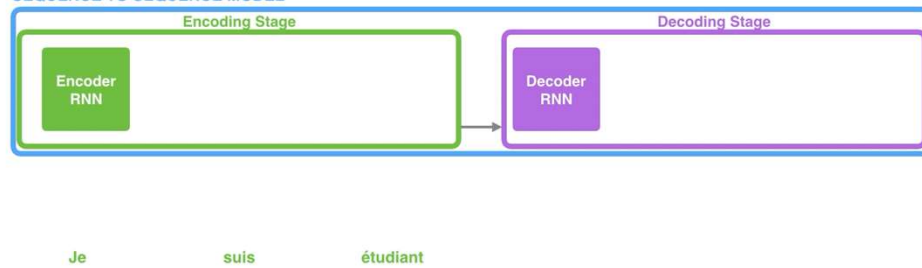


Source: <https://jalammar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/>

24

Hidden state travels through RNN cells of encoder

Neural Machine Translation SEQUENCE TO SEQUENCE MODEL



Source: <https://jalammar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/>

25

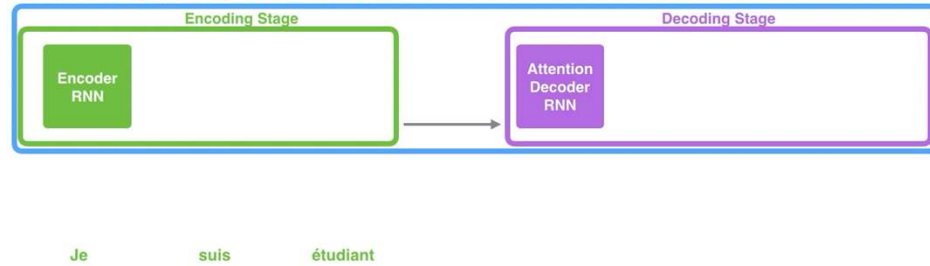
Issues with passing just the last hidden layer to decoder

1. Different part of decoder needs to focus on different part of encoder network
2. A single hidden layer (thought vector/hidden-state) becomes a **bottleneck** in the process , doesn't let decoder interact with all the encoder states individually
3. Least we can do is to pass all hidden state information instead of just the last hidden-state, that is where **attention** starts to come in the picture.

26

First thing different from simple encoder-decoder architecture

Neural Machine Translation SEQUENCE TO SEQUENCE MODEL WITH ATTENTION



Source: <https://jalammar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/>

27

Decoding at time step-7

Time step: 7

Neural Machine Translation SEQUENCE TO SEQUENCE MODEL WITH ATTENTION



Source: <https://jalammar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/>

28

Attention Decoder

an attention **decoder** does an extra step before producing its output. In order to focus on the parts of the input that are relevant to this decoding time step, the **decoder** does the following:

1. Look at the set of encoder **hidden states** it received – each **encoder hidden state** is most associated with a certain word in the input sentence
2. Give each **hidden state** a score (let's ignore how the scoring is done for now)
3. Multiply each **hidden state** by its softmaxed score, thus amplifying **hidden states** with high scores, and drowning out **hidden states** with low scores (*visuals in the next slide*)

Source: <https://jalammar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/>

29

**Second thing : Don't really pass all hidden states as is.
How do we compute attention?**

Attention at time step 4



Source: <https://jalammar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/>

30

Scoring?

1. Score or alignment score needs to be precise and can be computed in several ways
2. Scoring is done at each time step for the decoder
3. Below given is a table for some popular scoring functions

Name	Alignment score function	Citation
Content-base attention	$\text{score}(s_i, h_i) = \cos(\mathbf{s}_i, \mathbf{h}_i)$	Graves2014
Additive(*)	$\text{score}(s_i, h_i) = \mathbf{v}_a^T \tanh(\mathbf{W}_a [\mathbf{s}_i; \mathbf{h}_i])$	Bahdanau2015
Location-Base	$\alpha_{i,j} = \text{softmax}(\mathbf{W}_a \mathbf{s}_i)$ Note: This simplifies the softmax alignment to only depend on the target position.	Luong2015
General	$\text{score}(s_i, h_i) = \mathbf{s}_i^T \mathbf{W}_a \mathbf{h}_i$ where $\mathbf{W}_a$ is a trainable weight matrix in the attention layer.	Luong2015
Dot-Product	$\text{score}(s_i, h_i) = \mathbf{s}_i^T \mathbf{h}_i$	Luong2015
Scaled Dot-Product(*)	$\text{score}(s_i, h_i) = \frac{\mathbf{s}_i^T \mathbf{h}_i}{\sqrt{d_k}}$ Note: very similar to the dot-product attention except for a scaling factor; where n is the dimension of the source hidden state.	Vaswani2017

31

Looking Across the Decoder

This scoring exercise is done at each time step on the **decoder** side.

Let us now bring the whole thing together in the following visualization and look at how the attention process works:

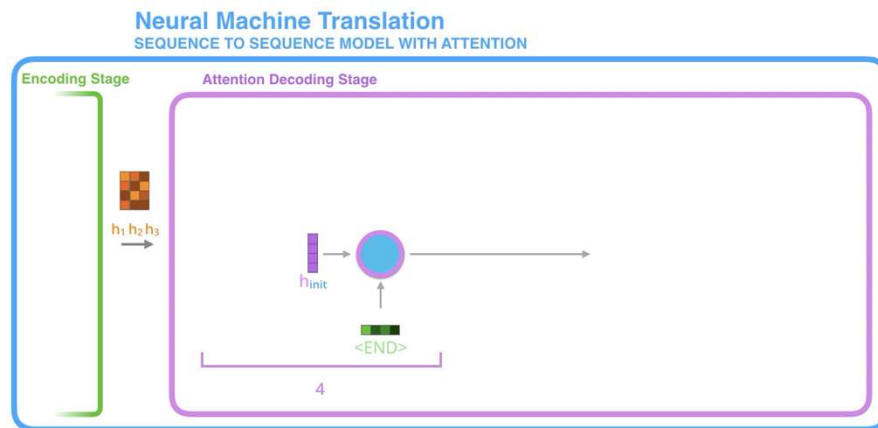
1. The attention decoder RNN takes in the embedding of the **<END>** token, and an **initial decoder hidden state**.
2. The RNN processes its inputs, producing an output and a **new hidden state vector (h4)**. The output is discarded.
3. Attention Step: We use the **encoder hidden states** and the **h4** vector to calculate a context vector (**C4**) for this time step.
4. We concatenate **h4** and **C4** into one vector.
5. We pass this vector through a **feedforward neural network** (one trained jointly with the model).
6. The **output** of the feedforward neural networks indicates the output word of this time step.
7. Repeat for the next time steps
(**visuals in the next slide**)

Source: <https://jalamar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/>

32

Looking Across the Decoder

Context vector and hidden-state information are concatenated together into a single attention vector.

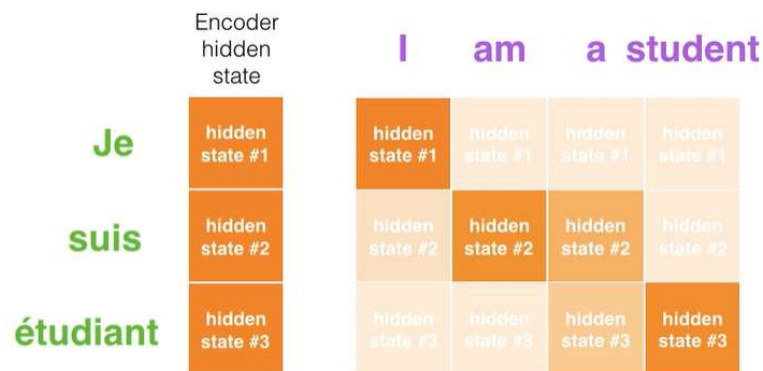


Source: <https://jalammar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/>

33

Understanding the Decoding Step

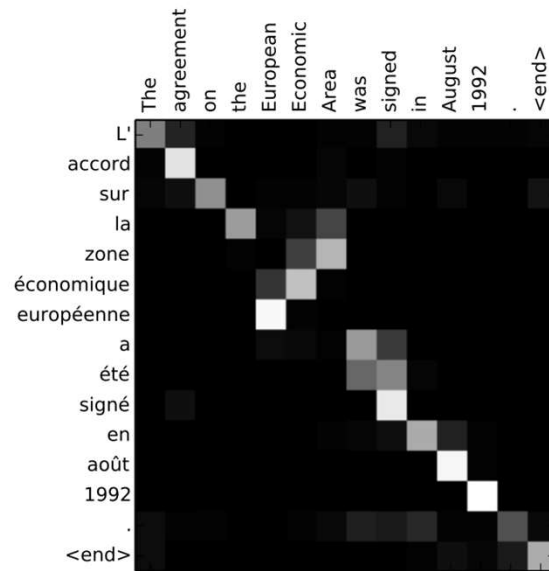
This is another way to look at which part of the input sentence we're paying attention to at each decoding step



Source: <https://jalammar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/>

34

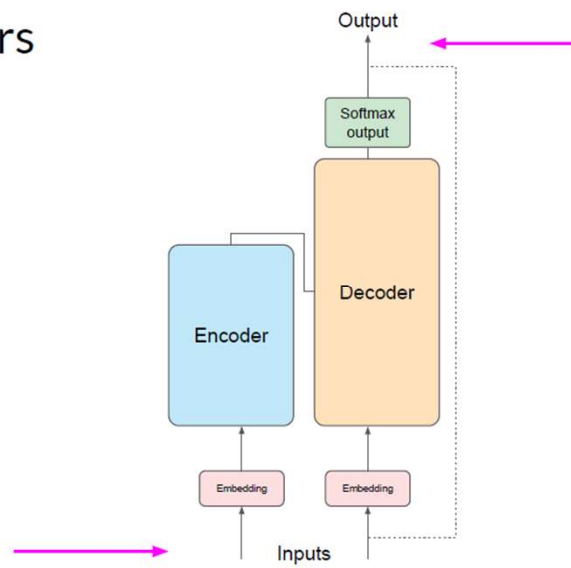
Paying Attention



Source: <https://jalammar.github.io/visualizing-neural-machine-translation-mechanics-of-seq2seq-models-with-attention/>

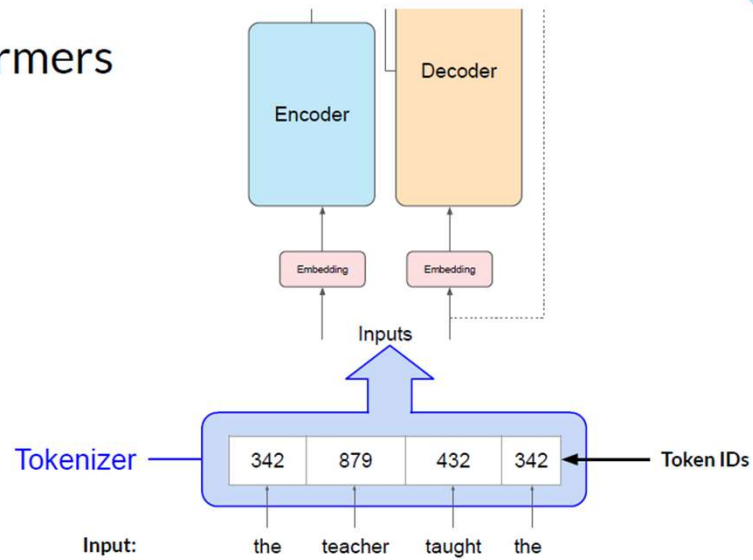
35

Transformers



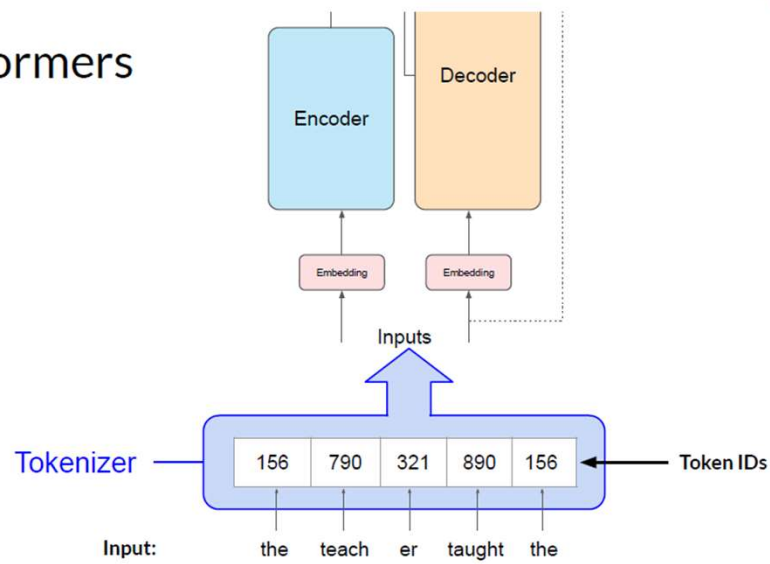
36

Transformers

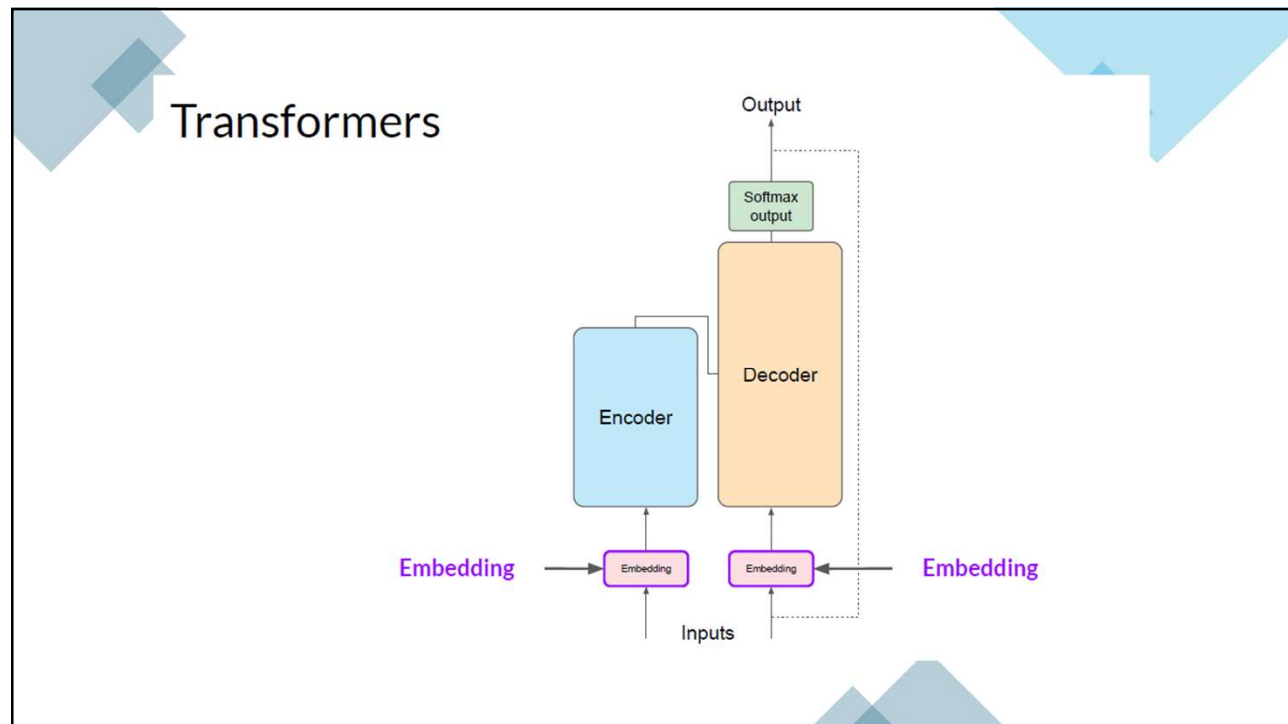


37

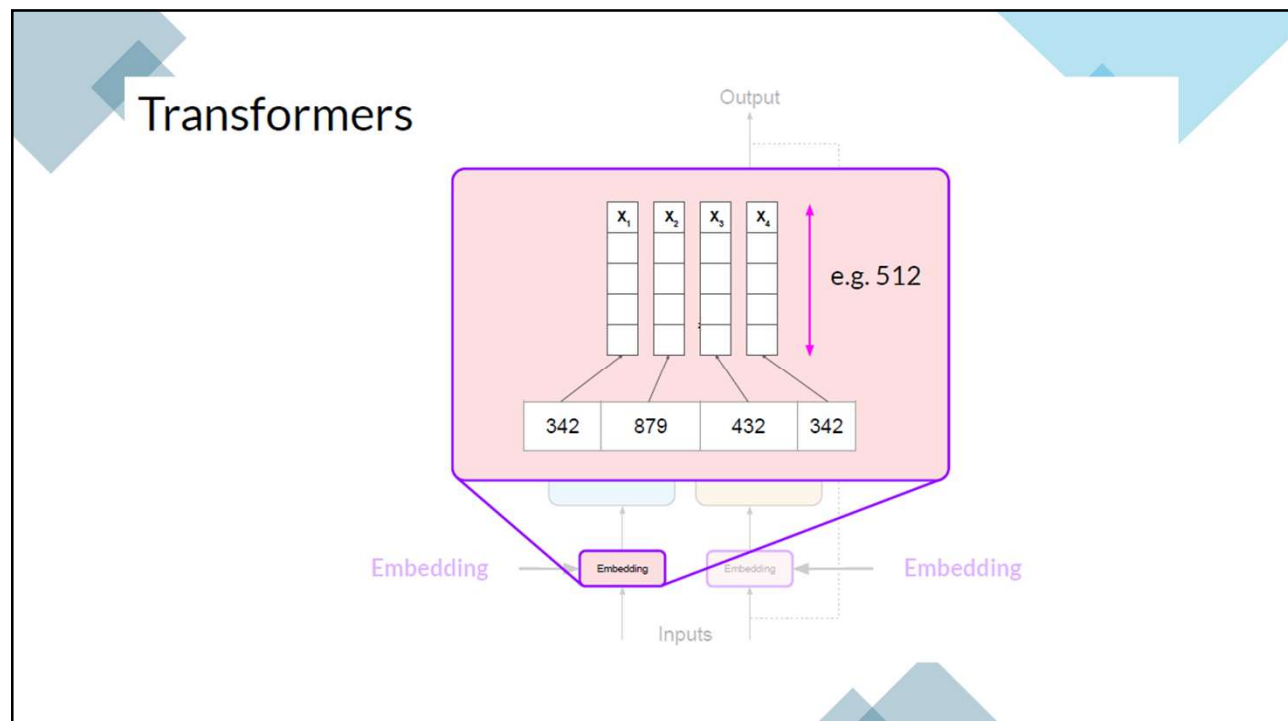
Transformers



38



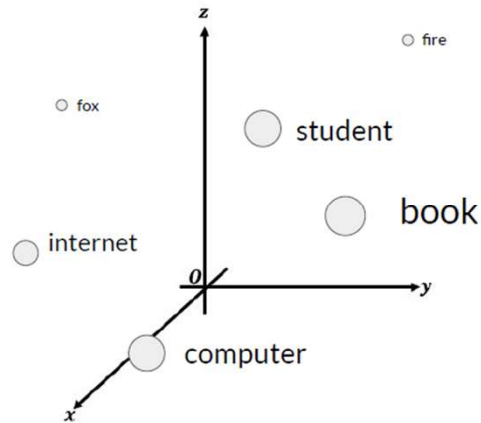
39



40

Transformers

Word
Embeddings

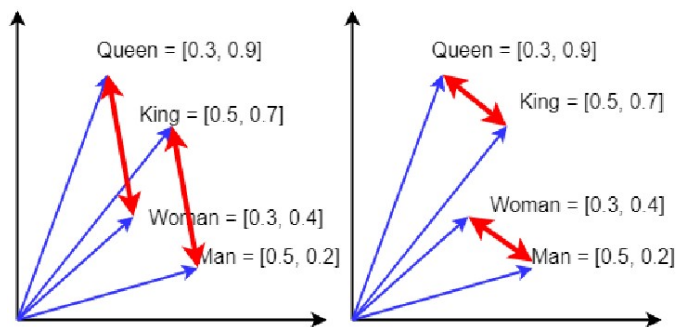


41

Vector Arithmetic on Word Vectors

The Big Idea of Word Embedding is to **turn text into numbers**.

This transformation is necessary because many machine learning algorithms (including deep nets) require their **input to be vectors of continuous values**; they just won't work on strings of plain text.

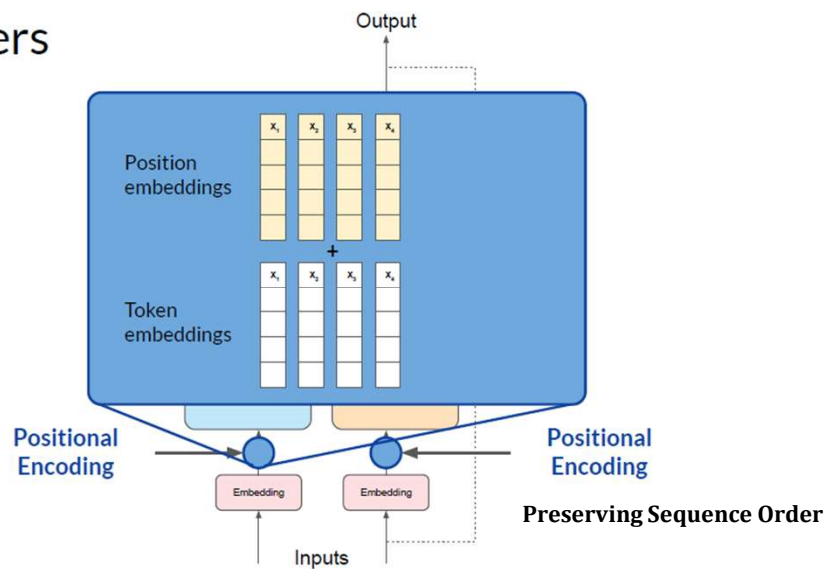


**King – Man + Woman =
Queen**

**[0.5, 0.7] – [0.5, 0.2] +
[0.3, 0.4]
= [0.3, 0.9]**

42

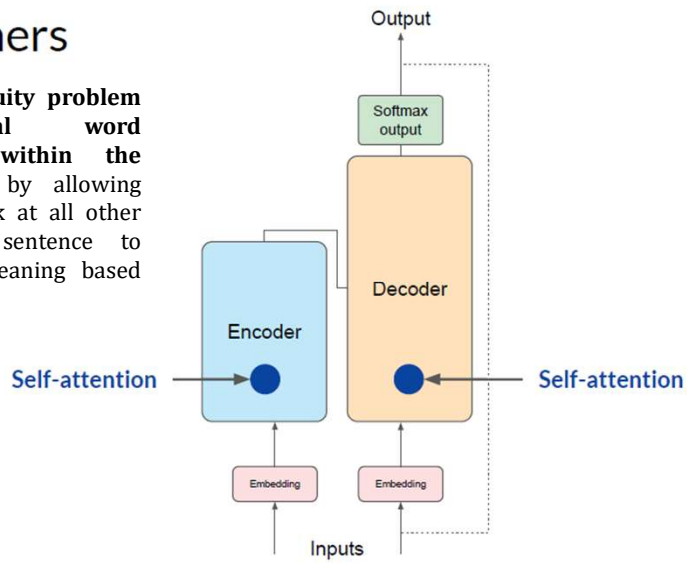
Transformers



43

Transformers

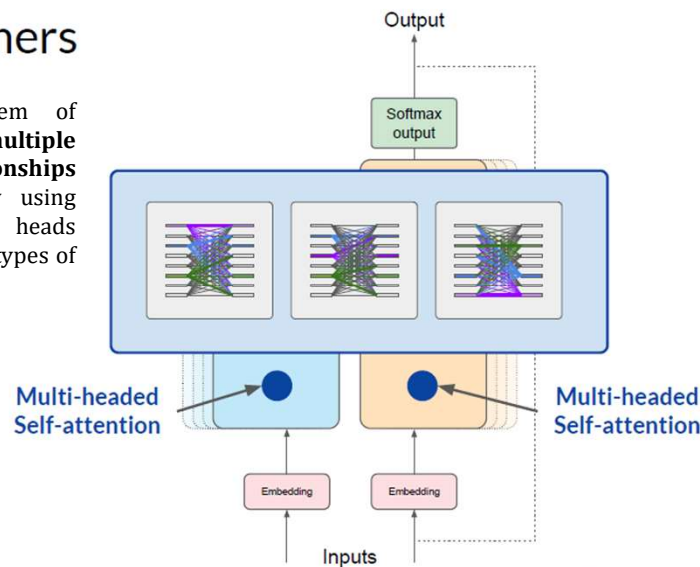
Solves the **ambiguity problem** of **contextual word representation within the same sequence** by allowing each word to look at all other words in the sentence to understand its meaning based on context.



44

Transformers

Solves the problem of **modeling multiple linguistic relationships simultaneously** by using parallel attention heads that learn different types of dependencies.



45

Concept	What problem does it solve?	Example: "The teacher taught the student with a book."
Attention	The Bottleneck Problem: Traditional models struggled to remember the beginning of a sentence by the time they reached the end. Attention creates a direct "shortcut" to the most relevant words.	When the model processes the word " book " at the end, Attention allows it to reach back and focus heavily on " taught " to understand that the book is the tool used for the action.
Self-Attention	The Ambiguity Problem: A word's meaning changes based on its neighbors. Self-attention allows every word in this specific sentence to "talk" to every other word to build context.	Self-attention calculates the relationship between " with a book " and " teacher ." It determines whether the "book" belongs to the "teacher" (the tool) or the "student" (the recipients) based on the surrounding syntax.
Multi-Head Attention	The Single-Perspective Problem: One "view" of a sentence isn't enough to capture grammar, meaning, and hierarchy all at once. Multi-head attention looks at the sentence through multiple "lenses" simultaneously.	Head 1 focuses on the subject-verb relationship (Teacher → Taught). Head 2 focuses on the object (Taught → Student). Head 3 focuses on the instrument used (Taught → Book).

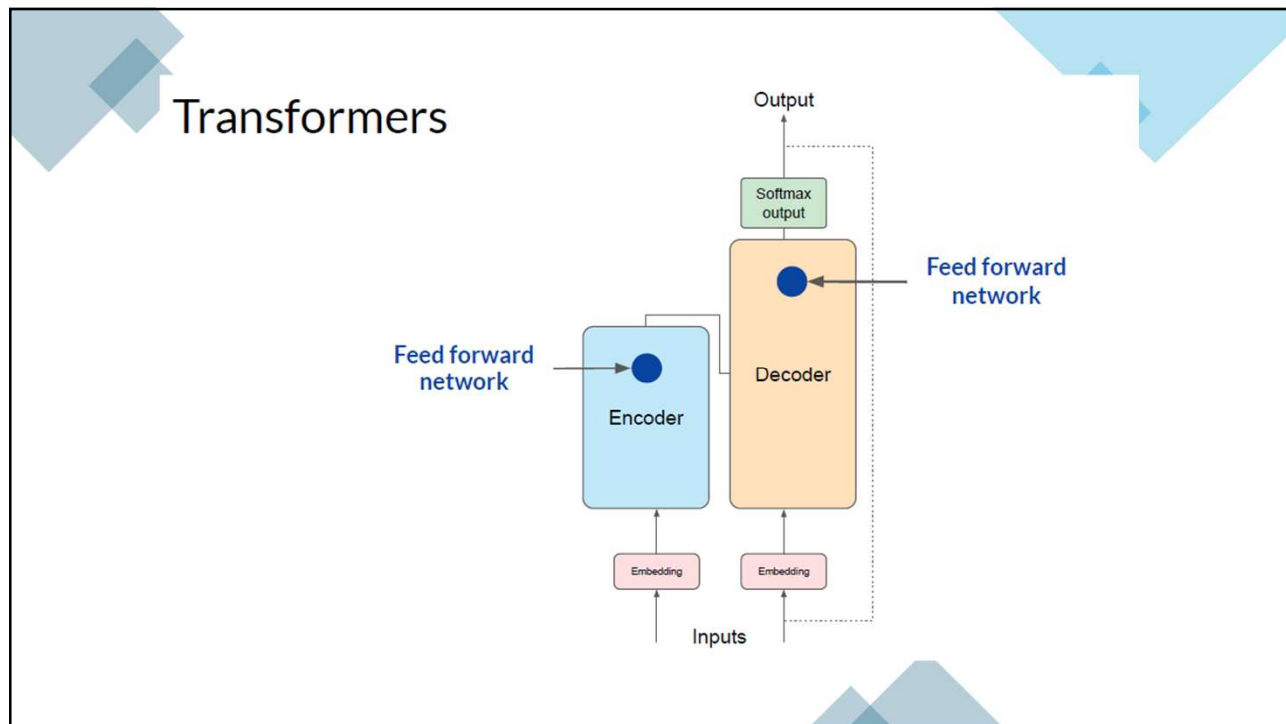
46

Concept	What problem does it solve?	Example sentence	How it helps in the example sentence
Attention	Solves the problem of information bottleneck in encoder-decoder models by allowing the decoder to dynamically access relevant parts of the input instead of relying on a single compressed vector.	<i>"Marie Curie won the Nobel Prize because she discovered radium."</i>	While generating " she ", the decoder assigns higher weight to " Marie Curie ", ensuring the pronoun correctly refers to her and not to "Nobel Prize" or "radium."
Self-Attention	Solves the problem of understanding word meaning based on full sentence context by allowing each token to interact with every other token in the same sentence.	<i>"He sat by the river bank and watched the water flow."</i>	The word " bank " attends to " river " and " water ", helping the model interpret it as a riverside location rather than a financial institution.
Multi-Head Attention	Solves the problem of modeling multiple linguistic relationships simultaneously by using parallel attention heads that learn different types of dependencies.	<i>"The scientist who developed the vaccine received global recognition."</i>	One head may focus on the long-range dependency (scientist → received), while another focuses on the relative clause (scientist → developed → vaccine), allowing the model to capture both structural and semantic relationships together.

47

Concept	What problem does it solve?	Example sentence	How it helps in the example sentence
Attention	Solves the problem of long-range dependency in sequence-to-sequence models by allowing the decoder to focus on the most relevant encoder words instead of compressing the entire sentence into a single fixed vector.	<i>"The animal didn't cross the street because it was too tired."</i>	While generating " it ", the decoder assigns higher weight to " animal " than to other words like <i>street</i> , ensuring the pronoun correctly refers to the animal and not something else.
Self-Attention	Solves the problem of contextual word representation within the same sequence by allowing each word to look at all other words in the sentence to understand its meaning based on context.	<i>"The bank approved the loan."</i>	The word " bank " attends to context words like " approved " and " loan ", helping the model infer the financial meaning instead of the river-bank meaning.
Multi-Head Attention	Solves the problem of capturing multiple types of relationships simultaneously by running several self-attention mechanisms in parallel, each learning different relational patterns.	<i>"The book that the professor recommended was fascinating."</i>	One attention head may focus on resolving the main clause (book → was fascinating), while another focuses on the relative clause (professor → recommended), enabling the model to capture both grammatical and semantic relationships at the same time.

48

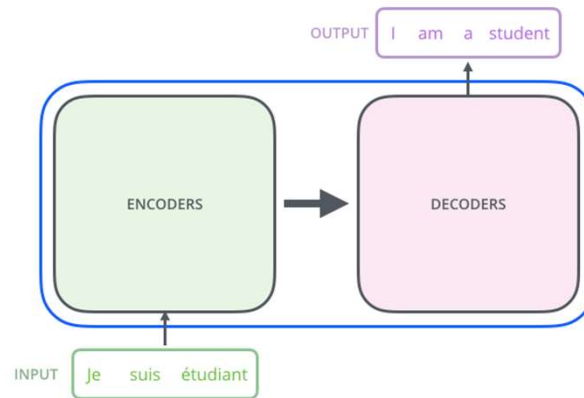


49

Transformer Architecture Inside Encoder/Decoder

50

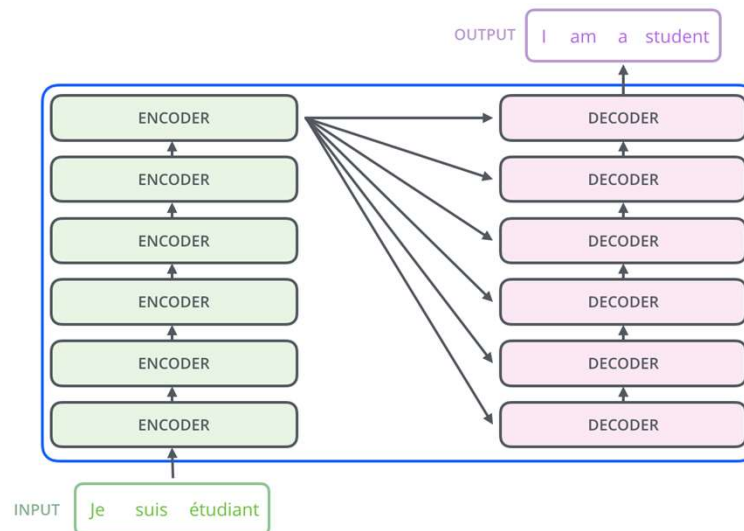
To start with, they use the same architecture



Source : <http://jalammar.github.io/illustrated-transformer/>

51

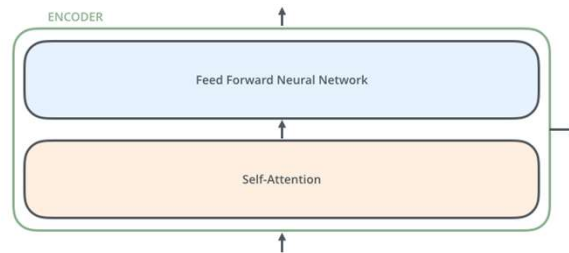
Encoder/Decoder are built differently



Source: <https://jalammar.github.io/illustrated-transformer/>

52

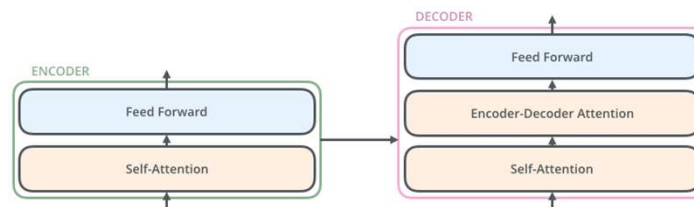
There is more inside these encoders



1. Encoders are all identical in structure
2. They don't share weights
3. Each one is broken into two sub layers as shown above

53

Connecting to the Decoder

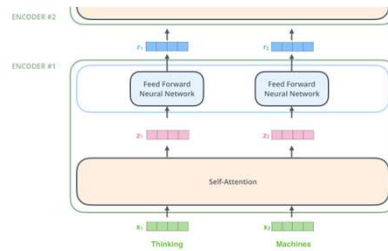


1. Self attention layer on encoder input lets the encoder look at other words as it encodes a specific word
2. Same FFN is applied to each position
3. Decoder has an additional attention layer to focus on different parts of the inputs coming from encoder

Source: <https://jalammar.github.io/illustrated-transformer/>

54

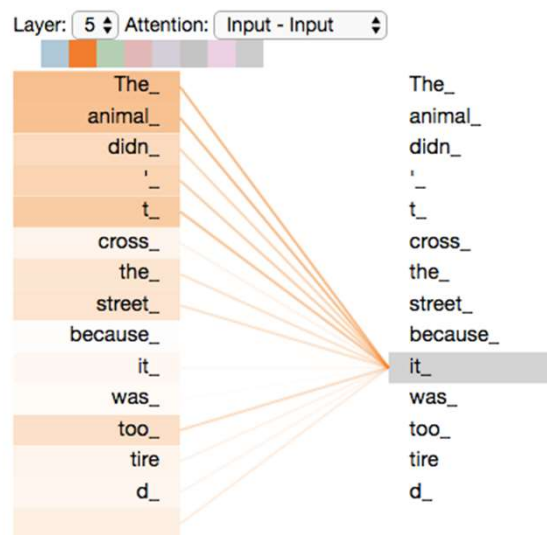
How input flows through the Encoder?



1. Embedding only happens right before the first encoder block, then onwards, encoder blocks pass output to the next encoder block as input.
2. Word in each position flows through its own path in encoder
3. There are dependencies in between these paths in the attention layer. That's not the case for FFN, hence it can be trained in parallel.

55

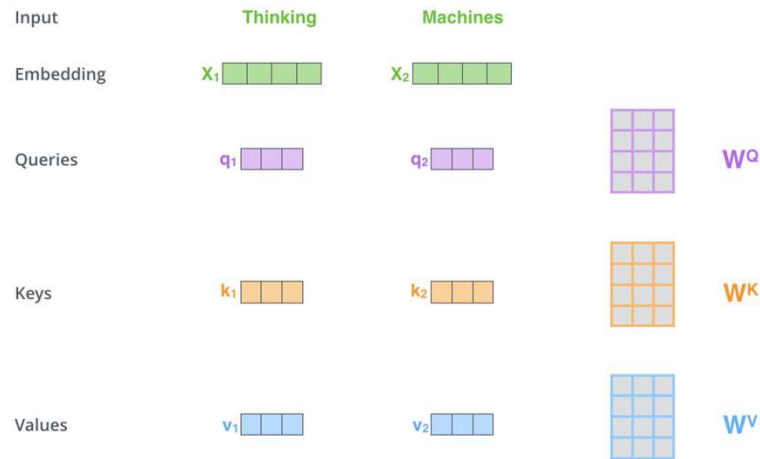
What exactly is this self-attention ?



Source: <https://jalammar.github.io/illustrated-transformer/>

56

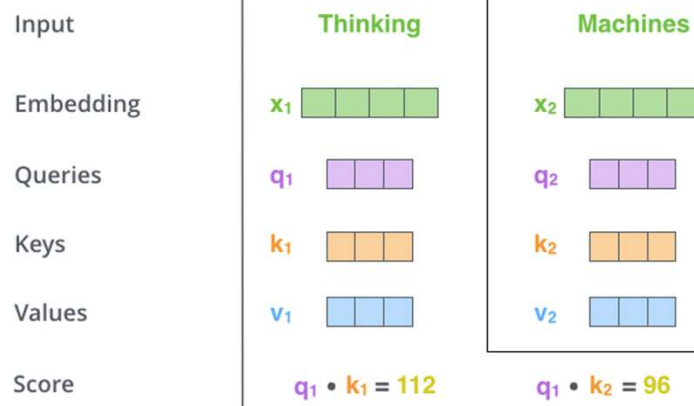
Components of self attention layer



Source: <https://jalammar.github.io/illustrated-transformer/>

57

Scoring across inputs for individual word



Source: <https://jalammar.github.io/illustrated-transformer/>

58

Scaling and Softmax

Input	Thinking	Machines
Embedding	x_1	x_2
Queries	q_1	q_2
Keys	k_1	k_2
Values	v_1	v_2
Score	$q_1 \cdot k_1 = 112$	$q_1 \cdot k_2 = 96$
Divide by $8 (\sqrt{d_k})$	14	12
Softmax	0.88	0.12

Source: <https://jalammargithub.io/illustrated-transformer/>

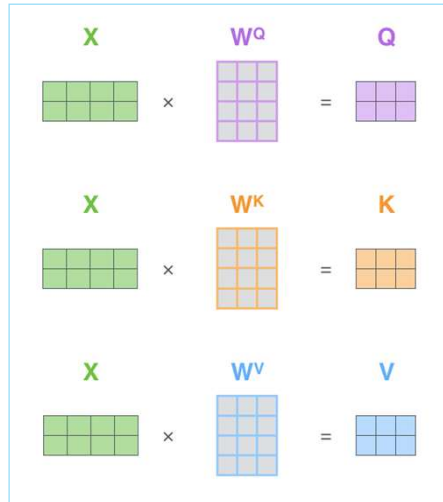
59

Self-Attention Output

Input	Thinking	Machines
Embedding	x_1	x_2
Queries	q_1	q_2
Keys	k_1	k_2
Values	v_1	v_2
Score	$q_1 \cdot k_1 = 112$	$q_1 \cdot k_2 = 96$
Divide by $8 (\sqrt{d_k})$	14	12
Softmax	0.88	0.12
Softmax X Value	v_1	v_2
Sum	z_1	z_2

60

A quick relook at matrix multiplication in self-attention



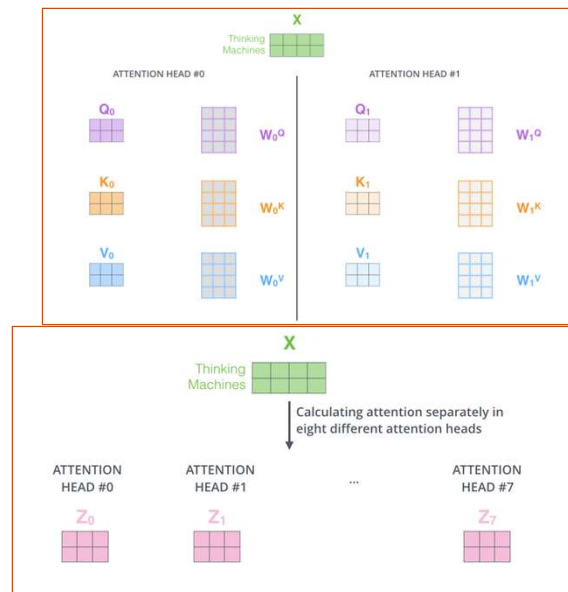
$$\text{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) \times V = Z$$

The diagram shows the calculation of the attention matrix Z. It involves the softmax operation applied to the scaled dot product of the Query matrix Q and the Key matrix K^T, followed by multiplication with the Value matrix V.

Source: <https://jalammar.github.io/illustrated-transformer/>

61

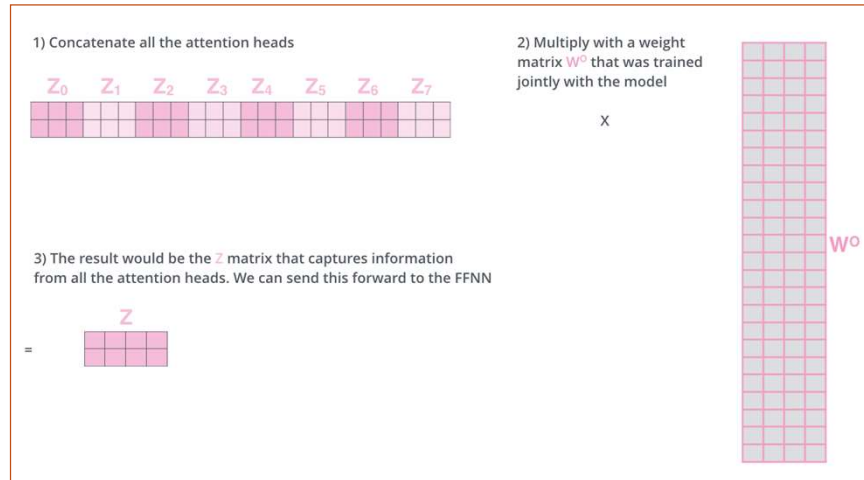
Multi-Head Attention



1. Original paper recommends 8 such heads
2. They generate 8 outputs

62

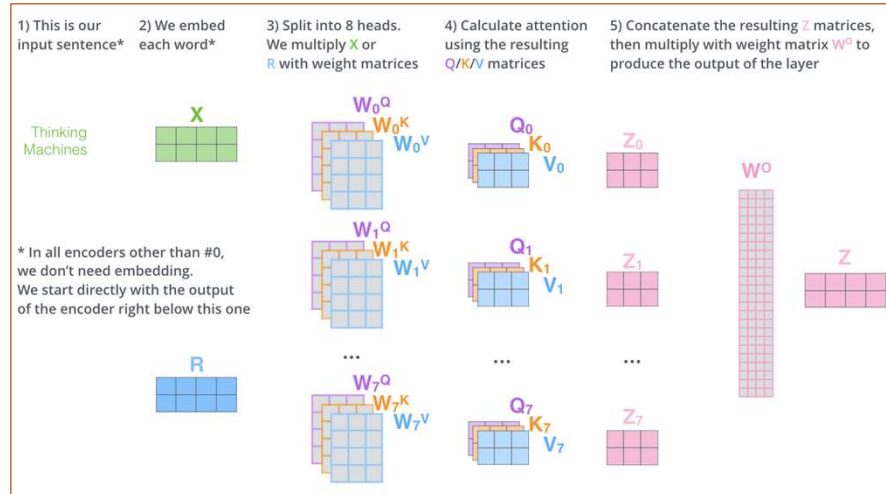
Concatenating and condensing outputs of multi-head



Source: <https://jalammar.github.io/illustrated-transformer/>

63

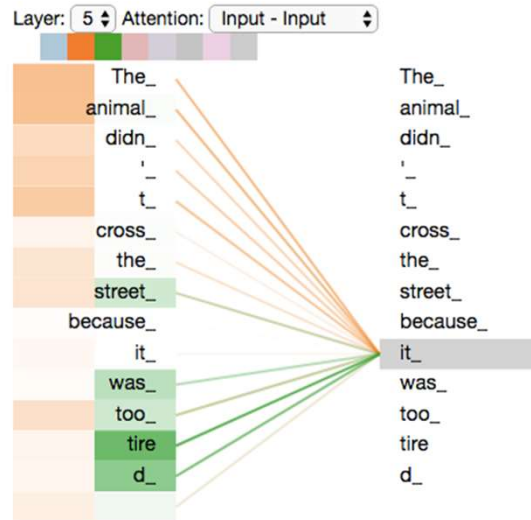
Overview



Source: <https://jalammar.github.io/illustrated-transformer/>

64

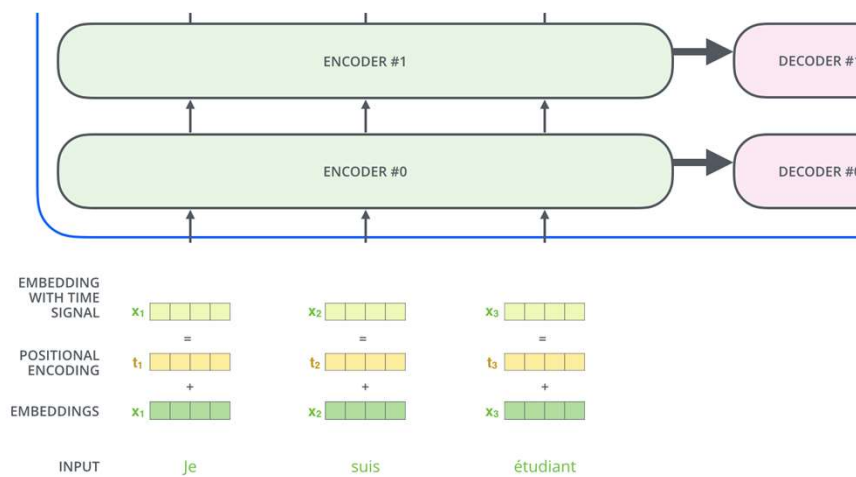
What does multi head attention look like?



Source: <https://jalammar.github.io/illustrated-transformer/>

65

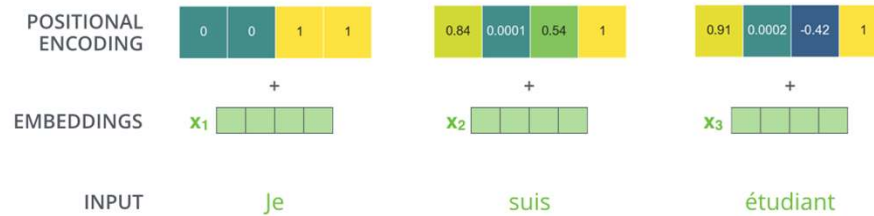
Preserving Sequence Order without RNNs



Source: <https://jalammar.github.io/illustrated-transformer/>

66

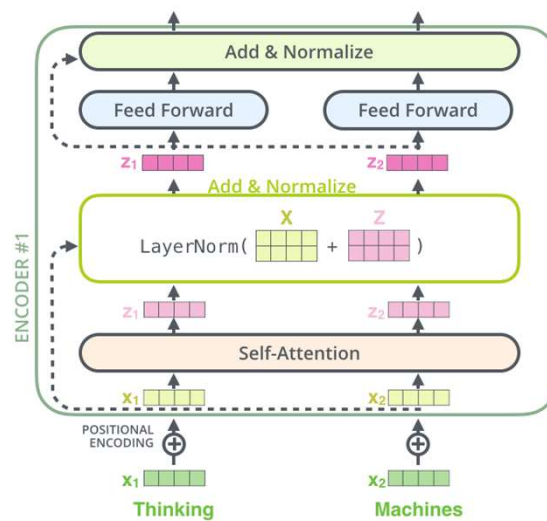
Positions are encoded with pairs of sin/cos alternatively



Source: <https://jalammargithub.io/illustrated-transformer/>

67

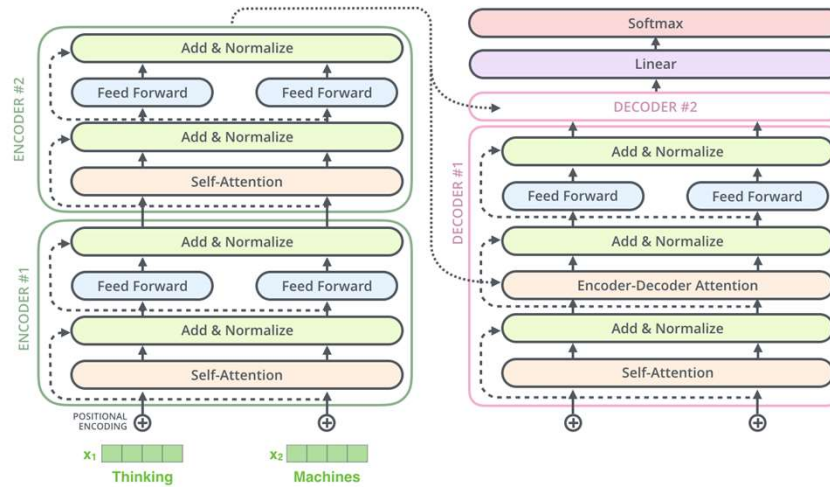
Residual connections



Source: <https://jalammargithub.io/illustrated-transformer/>

68

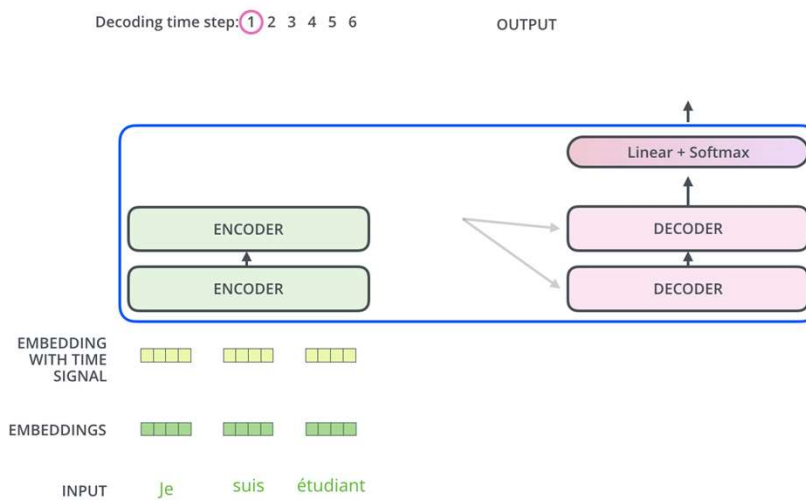
Residual connections are part of the Decoders as well



Source: <https://jalammargithub.io/illustrated-transformer/>

69

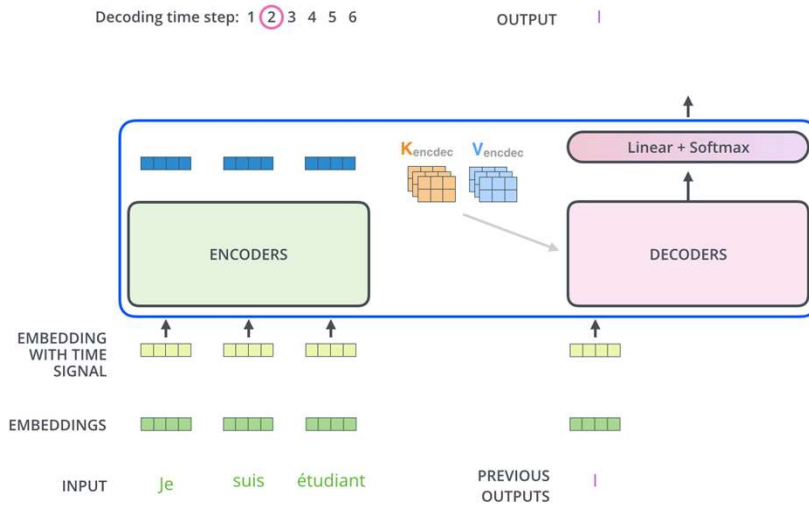
Process sequence with the Decoder



Source: <https://jalammargithub.io/illustrated-transformer/>

70

Process sequence with the Decoder (cont...)

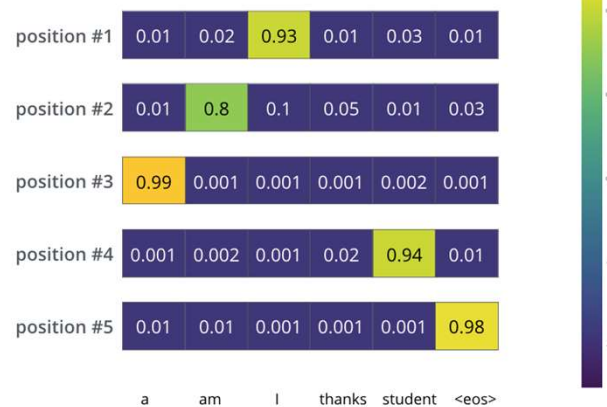


71

Target Model Outputs

Trained Model Outputs

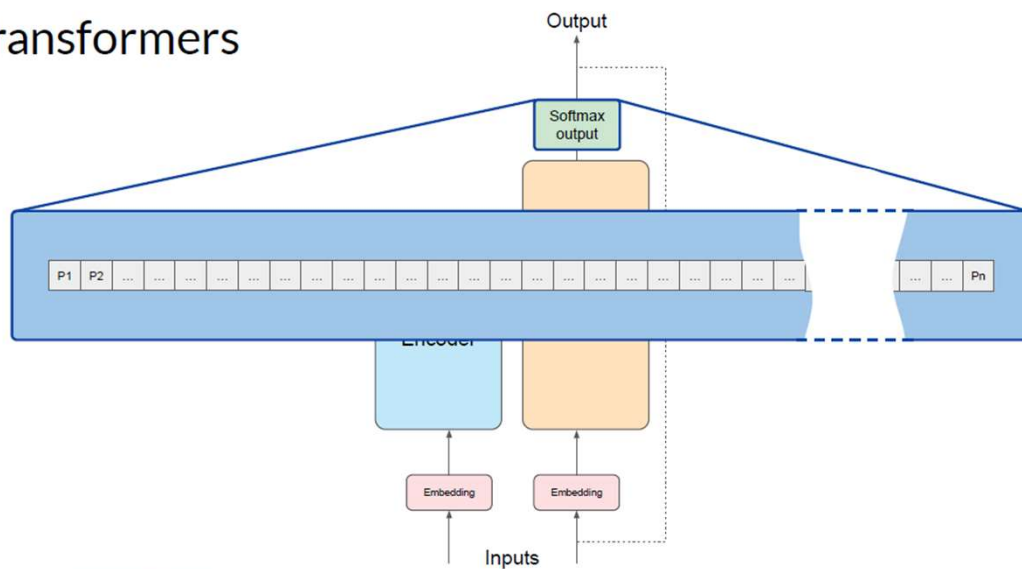
Output Vocabulary: a am I thanks student <eos>



Source: <https://jalammar.github.io/illustrated-transformer/>

72

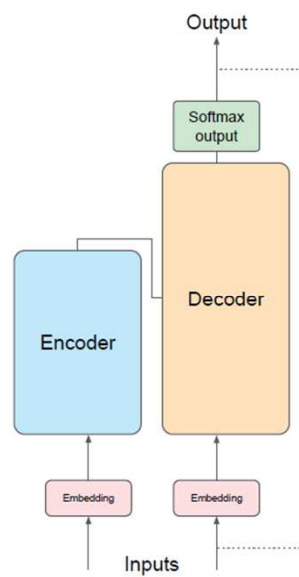
Transformers



73

Transformers

Encoder
Encodes inputs ("prompts") with contextual understanding and produces one vector per input token.

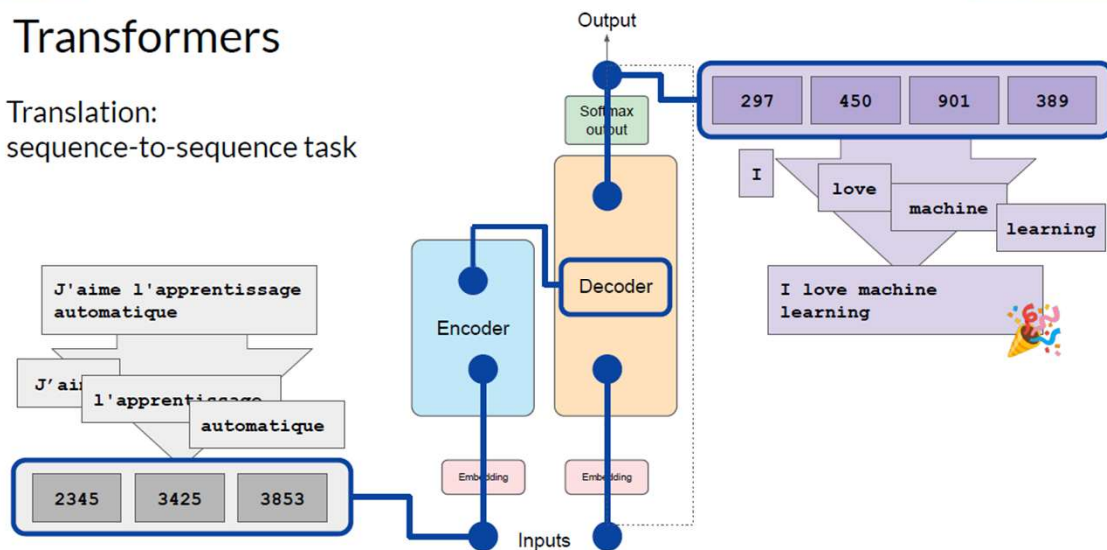


Decoder
Accepts input tokens and generates new tokens.

74

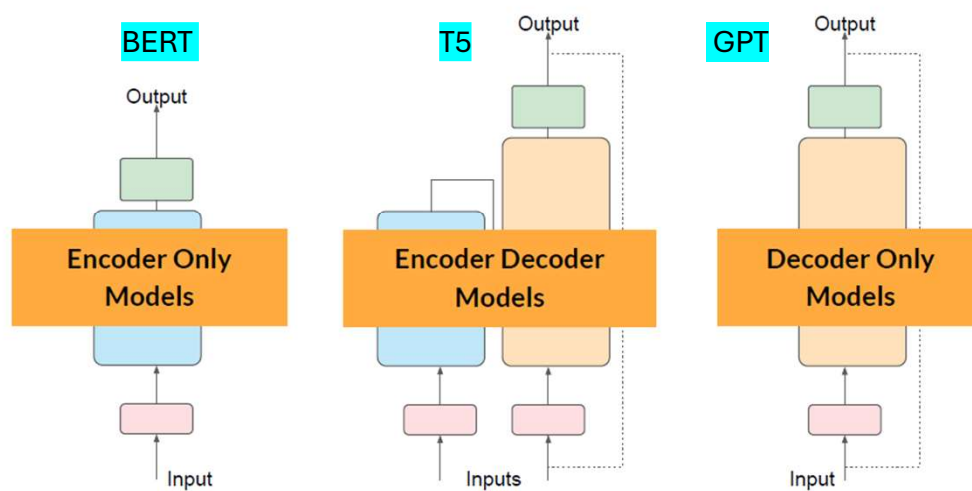
Transformers

Translation:
sequence-to-sequence task



75

Transformers



76

Architecture Type	What it is mainly used for	Example Models	Typical Task Example
Encoder-Only Transformer	Learns deep bidirectional contextual representations of input text	BERT, RoBERTa, DistilBERT	Text classification, sentiment analysis, named entity recognition
Decoder-Only Transformer	Autoregressively generates text by predicting the next token	GPT (GPT-2, GPT-3, GPT-4), LLaMA, Mistral	Text generation, chatbots, code generation
Encoder-Decoder Transformer	Encodes input sequence and generates a different output sequence	T5, BART, original Transformer (Vaswani et al.), MarianMT	Machine translation, summarization, paraphrasing

77

Context Aware Word Embeddings with BERT

(Bidirectional Encoder Representation from Transformers)

Bidirectional Encoder Representations from Transformers (BERT) by Devlin et al. (2019).
<https://arxiv.org/pdf/1810.04805.pdf>

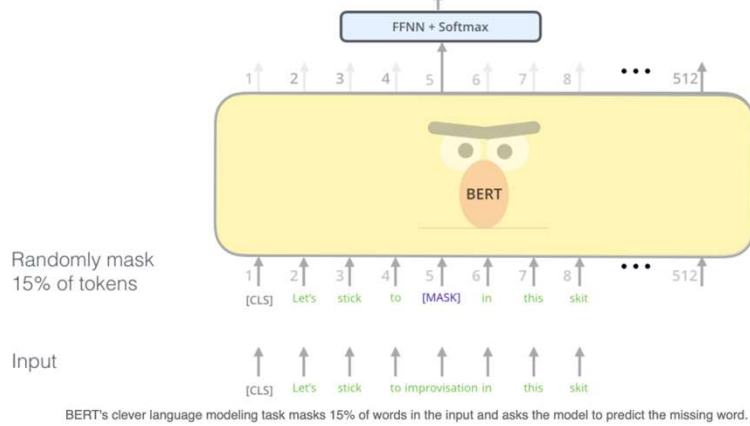
78

BERT's Tasks (done using Transformers)

Use the output of the masked word's position to predict the masked word

Possible classes:
All English words

0.1%	Aardvark
...	...
10%	Improvisation
...	...
0%	Zyzzzyva

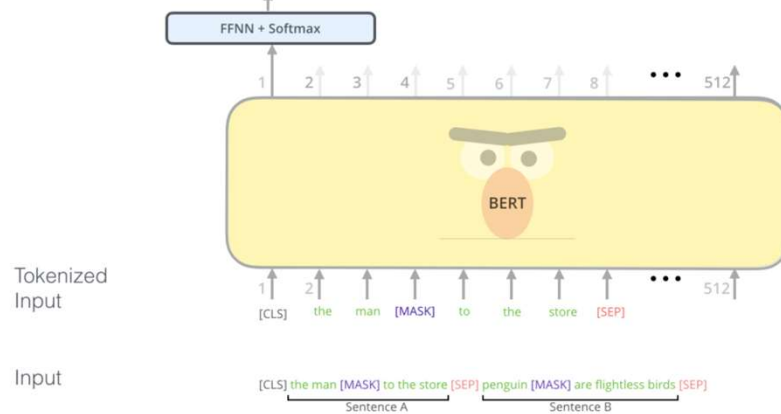


79

Two Sentence Task

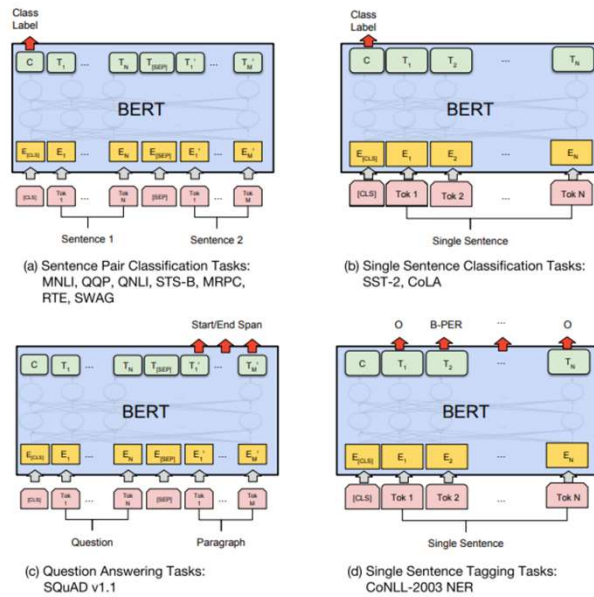
Predict likelihood that sentence B belongs after sentence A

1%	IsNext
99%	NotNext



80

Other Tasks



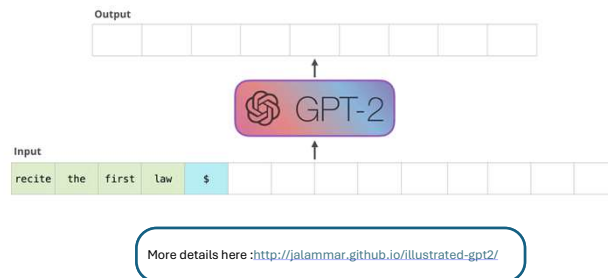
81

Context Aware Word Embeddings : GPT-2 (Generative Pretrained Transformer-2)

82

GPT-2

1. Language model which can predict next word in sequence given past words
2. Decoder only transformer architecture
3. Best text generation model so far



83

Attention in Vision Transformer (ViT)

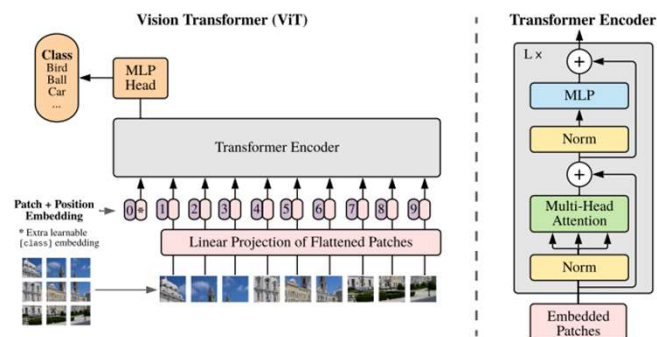


Figure 1: Model overview. We split an image into fixed-size patches, linearly embed each of them, add position embeddings, and feed the resulting sequence of vectors to a standard Transformer encoder. In order to perform classification, we use the standard approach of adding an extra learnable “classification token” to the sequence. The illustration of the Transformer encoder was inspired by Vaswani et al. (2017).

Vision Transformer: Dosovitskiy et al. (2020); in An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale
<https://paperswithcode.com/paper/an-image-is-worth-16x16-words-transformers-1>

84

References

- **Vaswani et al. (2017): Attention is all you need (introduction to Transformer)**
<https://papers.nips.cc/paper/7181-attention-is-all-you-need.pdf>
- **Bidirectional Encoder Representations from Transformers (BERT) by Devlin et al. (2019).**
<https://arxiv.org/pdf/1810.04805.pdf>
- **Vision Transformer: Dosovitskiy et al. (2020); in An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale**
<https://paperswithcode.com/paper/an-image-is-worth-16x16-words-transformers-1>

85

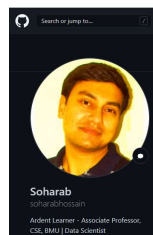
Links to Useful Resources

- **Text Extraction with BERT (Keras coding example)**
https://keras.io/examples/nlp/text_extraction_with_bert/
- **HuggingFace: Transformer**
https://huggingface.co/transformers/model_doc/bert.html#tfbertmodel
- **Github: Bert-Extractive-Summarizer**
<https://github.com/dmmiller612/bert-extractive-summarizer>
- **BERT Model**
https://huggingface.co/transformers/model_doc/bert.html
- **Github: Sentence-Transformers (to find sentence embeddings)**
<https://github.com/UKPLab/sentence-transformers>

86



87



Thank You

Class material will be uploaded here:
https://github.com/soharabhossain/GenAI_n_LLMs

88